

机器人代码迁移项目进度简报

Panda -> xArm6 / Fetch · ManiSkill · Adapter Generation Harness

1. 当前结论

当前已经从单次 LLM 生成 adapter，推进到一个 simulation-in-the-loop 的迁移 harness 原型。高层程序保持不变，LLM 主要生成 target-side adapter，由真实 ManiSkill 仿真验证是否成功。

最新验证结果：PullCube Panda -> xArm6、PickCube Panda -> xArm6、PullCube Panda -> Fetch 三个已注册迁移 case 均已跑通 success=True / ret_val=True。

边界说明：当前成果不是“任意任务任意机器人必然成功”，而是已注册 case 可以进入自动迁移闭环，并通过仿真反馈持续修复。

2. 三个迁移任务状态

任务	源机器人	目标机器人	状态	说明
PullCube-v1	Panda	xArm6 Robotiq	成功	接触式拉方块；adapter 处理接触点、拖拽阶段和 action 填充。
PickCube-v1	Panda	xArm6 Robotiq	成功	抓取式任务；adapter 处理抓取高度、闭合时序和放置流程。
PullCube-v1	Panda	Fetch	成功	移动底盘/9D action 更复杂；当前已作为成功迁移 case 验证。

3. 现在框架做了什么

- 保持高层 LMP 程序不变，只迁移目标机器人 adapter。

- adapter 把 robot.pull / robot.grasp / robot.place 翻译成真实 env.step(action) 序列。
- 失败后不是只看 success=False，而是记录失败层、诊断信息和 structured probe 结果。
- 新增 agent mode：从 seed adapter 开始，自动调用 LLM、仿真验证、probe、再修复。

4. 推荐展示命令

从零 agent 迁移入口：

```
python migrate.py --task pull_cube --source panda --target xarm6 --mode agent --max-cycles 5
```

单次验证当前 adapter：

```
python migrate.py --task pick_cube --source panda --target xarm6 --mode evaluate --seed 0
```

5. 代码对比图

下面的图是核心代码片段对比，用来说明“迁移前后变了什么”。完整代码仍以仓库中的 case_programs、generated_adapters、seed_adapters 为准。

Figure 1. High-level program vs target adapter

High-level LMP program	Target-side adapter responsibility
<pre>cube = scene.get_object("cube") goal = scene.get_region("goal") ret_val = robot.pull(cube, goal)</pre>	<pre>def pull(self, obj, target, ...): cube_pos = self._actor_pos("cube") goal_pos = self._region_pos(target.name) contact = choose_contact_pose(cube_pos, goal_pos) self._move_towards(pre_contact, gripper=-1) self._move_towards(contact, gripper=-1) for waypoint in staged_drag_path: action = self._make_action(delta_xyz, gripper=-1) self._step(action) return self._pull_cube_success()</pre>

图 1：高层程序很短，adapter 展开为具体执行序列

Figure 2. PullCube: contact/action execution changes

Panda/source assumption	Migrated target adapter pattern
<pre># Source-style PullCube primitive contact_x_offset_m = 0.07 contact_z_offset_m = 0.02 move_steps = 14 drag_steps = 42 # Fixed far-side contact contact = cube_pos + [x_offset, 0, z_offset] drag_end = [goal_x - extra, cube_y, contact_z] _move_towards(contact) _move_towards(drag_end)</pre>	<pre># Target adapter changes embodiment details _validate_action_space() _make_action(delta_xyz, gripper) # Contact geometry / timing tuned per robot contact_x_offset_m = robot_specific_value contact_z_offset_m = robot_specific_value max_delta_m = smaller_step stages = more_stable_drag # Progress checked by real ManiSkill success return self._pull_cube_success()</pre>

图2：PullCube 的主要变化在接触几何和 action 执行

Figure 3. PickCube: grasp becomes target-specific

High-level PickCube program	Migrated grasp adapter pattern
<pre>cube = scene.get_object("cube") goal = scene.get_region("goal") grasp_ok = robot.grasp(cube) ret_val = robot.place(cube, goal) if grasp_ok else False</pre>	<pre>def grasp(self, cube, ...): # adapter decides actual grasp envelope for z_offset in candidate_heights: pre_grasp = cube_pos + [0, 0, approach_h] grasp_pos = cube_pos + [0, 0, z_offset] self._move_towards(pre_grasp, gripper=open) self._move_towards(grasp_pos, gripper=open) self._repeat_action([0,0,0], gripper=close) if self._is_grasping("cube"): lift_and_place() return True</pre>

图3：PickCube 的主要变化在抓取 envelope 和闭合时序

Figure 4. One-shot generation vs agent repair loop

Earlier workflow	Current agent workflow
<pre>python -m maniskill_backend.module_generation_runner \ --case case02_pull_cube_panda_to_xarm6 \ --max-attempts 3 # One run gives: # LLM adapter -> simulator result # Human manually reads failure # Human adjusts prompt / reruns</pre>	<pre>python migrate.py \ --task pull_cube \ --source panda \ --target xarm6 \ --mode agent \ --max-cycles 5 # Loop: # restore seed adapter # LLM writes adapter # simulate and diagnose # structured probe if needed # LLM repair and retry</pre>

图 4：从一次性生成，变成 *simulation-in-the-loop agent repair*

6. 汇报时一句话总结

这两周我把机器人代码迁移从“手动 prompt + 单次实验”推进到“注册 case + LLM 生成 adapter + ManiSkill 验证 + structured probe + agent 循环修复”的框架。现在已有三个已注册迁移 case 跑通，下一步是扩大任务/机器人集合，并验证多 seed 泛化。